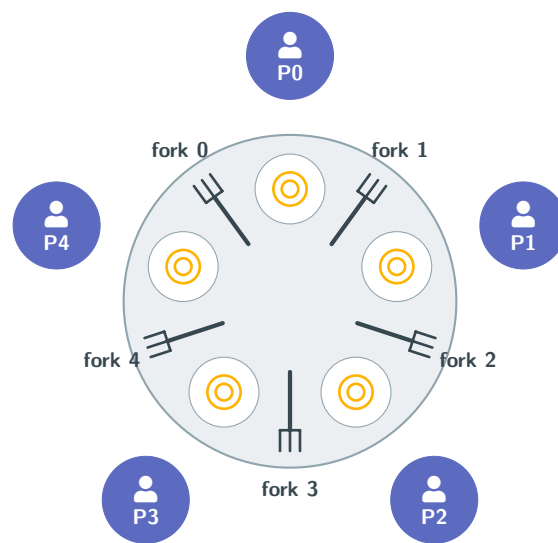


Task 11: The dining philosophers

Task 11 is about one of the most famous problems of concurrent programming. Read this introduction first.

The dining philosophers problem

In 1965 Edsger Dijkstra (the same person who described producer-consumer and invented semaphores) gave his students an exam problem about five computers that share five tape drives. A few years later Tony Hoare told the same problem as a story, and this story is how everybody knows it today.



Five philosophers sit at a round table. In front of each of them is a plate of spaghetti, and between every two neighbours lies **one fork**: five forks for five people. A philosopher does only two things, again and again: **thinks** and **eats**. To eat, a philosopher needs **both** forks: the one on the left and the one on the right. A fork can be held by only one philosopher at a time. After eating, the philosopher puts both forks back and thinks again.

Why is this important? The philosophers are threads, and the forks are resources that two threads share: locks, files, devices, database rows. Every program in which a thread needs **two or more** shared resources at the same time has the same problem. The table only makes it easy to see.

What can go wrong? The simplest rule is: “take the left fork, then the right fork”. Imagine that all five philosophers get hungry at the same moment, and each one takes the left fork. Now every fork is in someone’s hand, and every philosopher waits for the right fork, which the neighbour holds and will never put down. Nobody eats, ever. This is a **deadlock**, and you know its shape from Tasks 4 and 5: a **circle of waiting**, here all the way around the table.

There is a second danger: **starvation**. Even without a deadlock, a rule could be unfair, so that one philosopher almost never gets both forks while the neighbours eat again and again. A good solution has neither a deadlock nor starvation.

Deadlock and starvation

Deadlock: a group of threads waits in a circle, and none of them can ever go on.

Starvation: the program goes on, but one thread waits much longer than the others, or for ever.

The program

Every philosopher is a thread, and every fork is a `std::mutex`. Philosopher `i` has fork `i` on the left and fork `(i + 1) % 5` on the right. Everybody takes the left fork first. The dinner lasts 3s; then the program prints how many times each philosopher has eaten.

Ready code (in `table.h`, we do not look inside)

Functions

<code>think(id)</code>	think: 0 to 9 ms
<code>pick_up(fork)</code>	pick up a fork: lock its mutex
<code>eat(id)</code>	eat: 5 ms, and count the meal
<code>wait_for_end()</code>	wait 3s, then end the dinner
<code>report()</code>	print the number of meals of every philosopher

Constants and variables

<code>N</code>	5 philosophers and 5 forks
<code>dinner_over</code>	becomes true when the dinner ends

The template is in [Task11](#). This is `philosophers.cpp`:

```
std::mutex forks[N];           // fork i: between philosopher i - 1 and philosopher i

void philosopher(int id) {
    int left = id;             // the fork on my left
    int right = (id + 1) % N;  // the fork on my right
    while (!dinner_over) {
        think(id);
        pick_up(forks[left]);
        pick_up(forks[right]);
        eat(id);
        forks[right].unlock();
        forks[left].unlock();
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < N; id++)
        threads.emplace_back(philosopher, id);
    wait_for_end();
    for (auto &t : threads)
        t.join();
    report();
}
```

In the task folder, `make run` compiles the program and runs it.

Your tasks

- ❶ Run the program several times. Does it always end? (A program that hangs: stop it with Ctrl+C.)
- ❷ When it hangs, draw the table: which fork does each philosopher hold, and which one does each wait for? Where is the circle?
- ❸ **Fix 1:** one philosopher takes the forks in the other order. Rule: everybody takes the fork with the **smaller number** first. Which philosopher changes? Why does this break the circle?
- ❹ **Fix 2:** go back to the original code. A waiter lets at most **4** philosophers sit at the table at the same time; the fifth one waits until somebody stands up. Write it with a `std::counting_semaphore` (Task 9). Why are 4 enough to prevent the deadlock?
- ❺ Look at the numbers of meals in both fixes. Does anybody starve? Is one fix fairer than the other?